

WebGIS 瓦片地图—瓦片地图原理应用实战-如何实现地图切片工具

ThinkGIS 2022-08-01 3,859 阅读4分钟

关注

上篇文章介绍了地图瓦片加载计算的相关的原理，学习原理性的东西，到底有没有用，怎么用？日常工作中一般有成熟的第三方工具或者库可以实现。虽然很多时候是有工具的，大部分工具需要手动操作不能做到批量自化、动化操作，或者工具能力有限不能满足需求。作为技术同学不能只局限于使用工具，要学会创造工具，提供最优的解决方案。本系列教程教程语言实现均为 JS 语言，，读者需要具有 JS 开发基础并熟悉 Node。

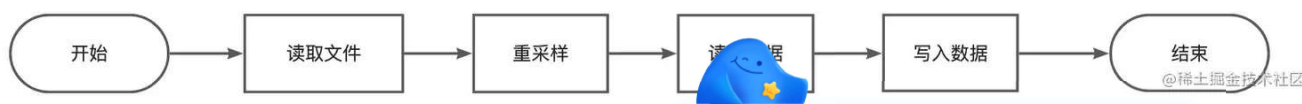
本文主要介绍如何实现地图瓦片切片工具，地图切片除了需要了解瓦片计算逻辑，还会涉及到数据投影、分辨率，重采样相关能力这里我们使用第三方库 Gdal 的node 版本 node-gdal。工具功能介绍可以看这篇文章

[Tiff 切片小工具，解决栅格数据可视化数据量大问题](#)

需求

遥感数据一般比较大，几百MB 或者几个 GB，如何在前端浏览器可视化确实是个问题，数据量不仅数据加载缓慢、而且地图渲染性能有很高的要求。瓦片是解决数据量大最好的方案，那问题怎么把一个 GeoTiff切成瓦片，目前没有合适的开源工具。

切片流程



读取文件

Gdal node 版本

登录掘金领取礼包
更多登录后权益等你解锁

登录 / 注册

naturalatlas.github.io/node-gdal/c...

node-gdal API Docs for Version: 0.11.0

Search for classes/modules...

Show: ☒ Inherited ☒ Protected ☐ Private ☒ Deprecated

Constants (CPL Error Codes)
Constants (CPL Error Levels)
Constants (DCAP)
Constants (DMD)
Constants (GCI)
Constants (GDT)
Constants (GRA)
Constants (ODsC)
Constants (OFT)
Constants (OJ)
Constants (OLC)
Constants (wkbByteOrder)
Constants (wkbGeometryType)
Constants (wkbVariant)
gdal
gdal.CoordinateTransformation
gdal.Dataset
gdal.DatasetBands
gdal.DatasetLayers
gdal.Driver
gdal.Envelope
gdal.Envelope3D
gdal.Feature
gdal.FeatureDefn
gdal.FeatureDefnFields
gdal.FeatureFields
gdal.FieldDefn
gdal.GDALDrivers

gdal.Driver

Format specific driver.

An instance of this class is created for each supported format, and manages information about the format.

This roughly corresponds to a file format, though some drivers may be gateways to many formats through a secondary multi-library.

Table of Contents

- Methods
 - CopyFiles(name_old, name_new)
 - createFilename(x, size, y, size, band, count, data_type, creation_options)
 - createCopy(filename, src, strict, options)
 - deleteDataset(filename)
 - getMetadata(domain)
 - open(path, mode)
 - rename(new_name, old_name)
- Attributes
 - description

Methods

copyFiles (name_old, name_new)
Defined in src/gdal_driver.cpp:385

@稀土掘金技术社区

读取文件为GeoTiff 文件, 这里使用 Gdal 进行读取。

▼ javascript

体验AI代码助手 复制代码

```

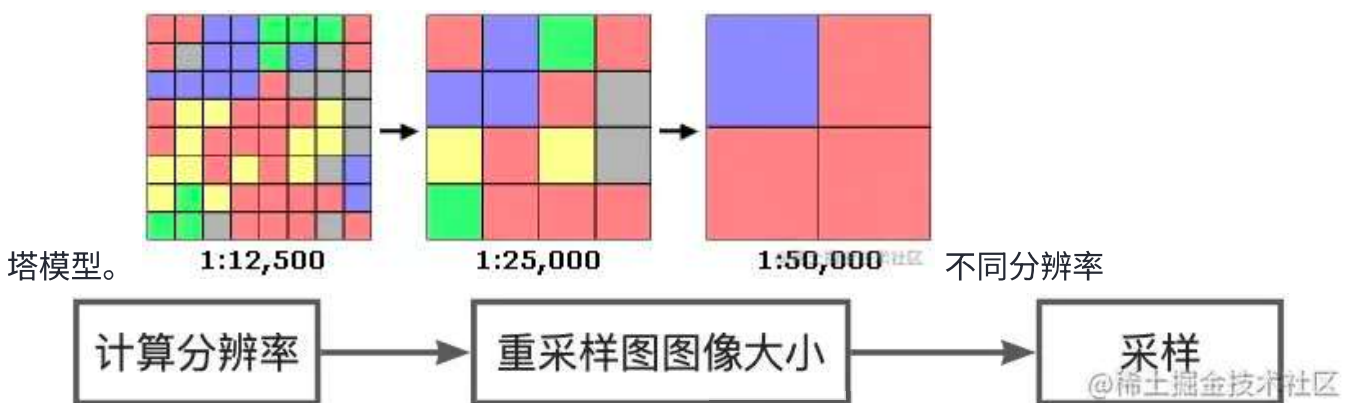
1 // 引入node-gdal 库
2 var gdal = require("gdal-next")
3 var dataset = gdal.open("../landcover_JiangXi.tif");

```

读取到文件 Gdal 提供了一些方法获取栅格影像的相关信息如坐标系、大小、分辨率、数据等等

重采样生成影像金字塔

为什么需要重采样, 瓦片切片需要根据地图缩放层级、生成不同分辨率的影像也就生成瓦片金字



分辨率计算

地球周长除以地图像素大小即为分辨率, 地球半径

$$resolution = C(\text{地球周长}) / Size(\text{地图像素大小})$$


登录掘金领取礼包

更多登录后权益等你解锁

享级相关

- 初始分辨率第 0 层级 一张 $256 * 256$ 的瓦片即可包含整个世界。

▼ javascript

 体验AI代码助手 

复制代码

```
1 var initialResolution = 2 * Math.PI * 6378137 / 256
```

- 各层级分辨率计算

▼ javascript

 体验AI代码助手 

复制代码

```
1 var resolution = initialResolution / Math.pow(2, N)
2
```

重采样大小

- width、height 原栅格数据长、宽
- xSize、ySize 原栅格数据分辨率
- resolution 目标分辨率

▼ javascript

 体验AI代码助手 

复制代码

```
1 const x = Math.round(width * xSize / resolution),
2 const y = Math.abs(Math.round(height * ySize / resolution))
```

相关参数计算完成之后, 我们就可以进行重采样了, 这里可以使用 `reprojectImage` 方法。

▼ javascript

 体验AI代码助手 

复制代码

```
1 reprojectRaster(dataset, zoom, path) {
2     const driver = dataset.driver;
3     const tileHelp = this.tileHelper;
4     const resolution = tileHelp.resolution(zoom);
5     const rasterSize = dataset.rasterSize;
6     const newSize = this.scaleZoomSize(rasterSize.x, rasterSize.y, dataset.geoTransform[1], (
7     const outDs = driver.create(path, newSize.x, newSize.y, 1, this.getDataType());
8     const gcp = dataset.getGCPs();
9     const gcpProject = dataset.srs.toWKT();
10    const [x, xtr, xr, yx, yr, ytr] = dataset.geoTransform;
11    outDs.geoTransform = [x, resolution, xr,
12    outDs.setGCPs(gcp, gcpProject);
13    outDs.bands.get(1).colorInterpretation = 1;
14    const option = {
15        src: dataset,
16        dst: outDs,
17        s_srs: dataset.srs,
```

 登录掘金领取礼包

更多登录后权益等你解锁

✕

on;

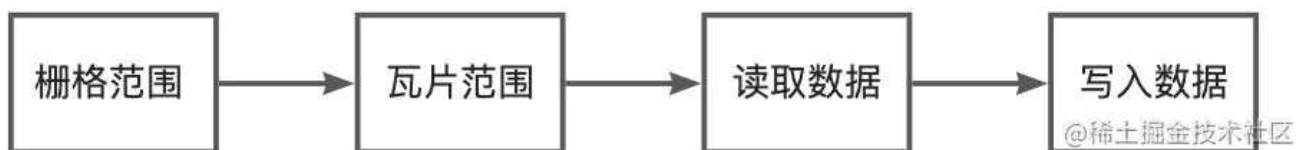
```

18         t_srs: dataset.srs,
19         resampling: gdal.GRA_NearestNeighbor,
20
21     };
22     gdal.reprojectImage(option)
23     outDs.flush();
24     outDs.close();
25
26 }

```

瓦片切片-平面分块

重采样之后,我们得到了不同分辨率的栅格影像,接下来就可以根据规则进行切片了。



栅格数据范围

栅格数据的经纬度范围直接 Gdal 可以读取,范围主要用于计算数据包含的瓦片范围进而进行切片。原始数据坐标系一般不是经纬度坐标系,需要进行坐标转换。原始坐标范围计算, ds.geoTransform 具体参数如下。

```

/*
六个参数分别是:
geos[0] top left x 左上角x坐标
geos[1] w-e pixel resolution 东西方向像素分辨率
geos[2] rotation, 0 if image is "north up" 旋转角度, 正北向上时为0
geos[3] top left y 左上角y坐标
geos[4] rotation, 0 if image is "north up" 旋转角度, 正北向上时为0
geos[5] n-s pixel resolution 南北向像素分辨率
x/y为图像的x/y坐标, geox/geoy为对应的投影坐标
*/
geox = geos[0] + geos[1] * x + geos[2] * y;
geoy = geos[3] + geos[4] * x + geos[5] * y

```

@稀土掘金技术社区

geoTransform坐标转换

▼ javascript

```

1  getRasterExtent() {
2      const ds = this.dataset
3      const size = ds.rasterSize
4      const minCorner = { x: 0, y: size.y };

```



登录掘金领取礼包

更多登录后权益等你解锁



制代码

```

5      const maxCorner = { x: size.x, y: 0 };
6      const geotransform = ds.geoTransform;
7      const wgs84 = gdal.SpatialReference.fromEPSG(4326);
8      const coord_transform = new gdal.CoordinateTransformation(ds.srs, wgs84);
9      const min = {
10         x: geotransform[0] + minCorner.x * geotransform[1] + minCorner.y * geotransform[2],
11         y: geotransform[3] + minCorner.x * geotransform[4] + minCorner.y * geotransform[5]
12     }
13     const max = {
14         x: geotransform[0] + maxCorner.x * geotransform[1] + maxCorner.y * geotransform[2],
15         y: geotransform[3] + maxCorner.x * geotransform[4] + maxCorner.y * geotransform[5]
16     }
17     const minWgs84 = coord_transform.transformPoint(min);
18     const maxWgs84 = coord_transform.transformPoint(max);
19     return [minWgs84.y, minWgs84.x, maxWgs84.y, maxWgs84.x]
20 }

```

瓦片范围计算

瓦片范围计算就需要了解瓦片技术规则了, 经纬度坐标如何转瓦片坐标, 如何计算参照[上篇文章](#)

▼ javascript

 体验AI代码助手

复制代码

```

1  boundsToTileExtent(minLon, minLat, maxLon, maxLat, zoom) {
2      const [minTx, minTy] = this.lonLatToTile(minLon, maxLat, zoom);
3      const [maxTx, maxTy] = this.lonLatToTile(maxLon, minLat, zoom);
4      return [[minTx, minTy], [maxTx, maxTy]]
5  }

```

读取数据

瓦片行列号范围计算好之后我们就可以根据行列号逐个读取瓦片的数据了。读取数据需要处理为边缘的瓦片, 数据超出区域的情况。

▼ javascript

 体验AI代码助手

复制代码

```

1  // 读取指定起点的瓦片数据
2  readRaster(origin) {
3      const tileHelp = this.tileHelper;
4      const band = this.dataset.bands.get(1);
5      const rasterSize = this.dataset.rasterSize;
6      const start = [Math.min(Math.max(0, origin[0], rasterSize - 1), rasterSize - 1),
7                     Math.min(Math.max(0, origin[1], rasterSize - 1), rasterSize - 1)];
8      const end = [Math.min(origin[0] + 256, rasterSize), Math.min(origin[1] + 256, rasterSize)];
9      const x = origin[0] < 0 ? -origin[0] : 0;

```



登录掘金领取礼包
更多登录后权益等你解锁



```

9      const y = origin[1] < 0 ? - origin[1] : 0;
10     const height= end[1] -start[1]
11     const width= end[0] -start[0]
12     const data = Array.from(band.pixels.read(start[0], start[1], width, height))
13     return {
14         data,
15         x,
16         y,
17         height,
18         width
19     }
20
21 }

```

写入数据

写入数据也需要注意, 在边缘的瓦片数据, 数据并不是完整的, 需要计算数据写入起始坐标。

▼ javascript

 体验AI代码助手

[复制代码](#)

```

1  writeTiffTile(x, y, z, data) {
2      const dataset = this.dataset;
3      const driver = dataset.driver;
4      const tileHelp = this.tileHelper;
5      if (!fs.existsSync(`${this.outPath}/${z}`)) {
6          fs.mkdirSync(`${this.outPath}/${z}`)
7      }
8      if (!fs.existsSync(`${this.outPath}/${z}/${x}`)) {
9          fs.mkdirSync(`${this.outPath}/${z}/${x}`)
10     }
11     const outDs = driver.create(`${this.outPath}/${z}/${x}/${y}.tiff`, tileHelp.tileSize, ti
12     const gcp = dataset.getGCPs();
13     const gcpProject = dataset.srs.toWKT();
14     const resolution = tileHelp.resolution(z);
15     const originMeters = tileHelp.pixelsToMeters(x * 256, y * 256, z)
16     const geoTransform = [originMeters[0], resolution, 0, originMeters[1], 0, -resolution];
17     outDs.geoTransform = geoTransform;
18
19     outDs.setGCPs(gcp, gcpProject)
20     const outBand = outDs.bands.get(1);
21     outBand.colorInterpretation = dataset.band
22     outBand.noDataValue = this.noDataValue;
23     outDs.srs = dataset.srs;
24     outBand.fill(this.noDataValue);
25     // TODO 数据类型
26     outBand.pixels.write(data.x, data.y, data

```



登录掘金领取礼包
更多登录后权益等你解锁



标签：

前端

GIS

数据可视化

评论 4



登录 / 注册 即可发布评论!

最热 最新



sugusGeek web前端开发
是不是想用江西地图做什么环江西的视频

1年前 点赞 评论

...



梨花猫 Golang @稀土掘金
golang 没有这个函数可以用什么替换 gdal.reprojectImage(option)

2年前 点赞 2

...



FinnNeo : 请问有没有可以实现webGIS地图切片的python库?

2年前 点赞 回复

...



梨花猫 回复 FinnNeo : python这种多的很啊

2年前 点赞 回复

...

目录

收起 ^

需求

切片流程

- 读取文件
 - Gdal node 版本
- 重采样生成影像金字塔
 - 分辨率计算
 - 重采样大小



登录掘金领取礼包
更多登录后权益等你解锁



瓦片切片-平面分块

栅格数据范围

瓦片范围计算

读取数据

写入数据

数据可视化

附录

相关推荐

Leaflet.js上高性能渲染百万级GeoJson数据的解决方案(无需服务端)

1.9k阅读 · 5点赞

最全最新cn中国【省、市、区县、乡镇街道】json, csv, sql数据

20k阅读 · 65点赞

Cesium+Vue实战教程——地图导航

4.0k阅读 · 28点赞

【三维GIS可视化】基于Vue+Cesium+Supermap实现智慧城市（三）

12k阅读 · 51点赞

cesium-2.加载天地图服务（影像和标注）

5.0k阅读 · 5点赞

精选内容

Vue 基础：状态管理入门

桜吹雪 · 56阅读 · 1点赞

程序设计

yuanyxh · 45阅读 · 0点赞

前端性能优化利器：LitePage 轻量级全页设计解析

eason_fan · 97阅读 · 1点赞

react-nil 逻辑渲染器

fe小陈 · 32阅读 · 0点赞

深入浅出贝塞尔曲线

布列瑟农的星空 · 120阅读 · 1点赞



登录掘金领取礼包

更多登录后权益等你解锁



为你推荐

探寻电子地图背后的秘密：瓦片技术的原理与实现

dreammy23 2年前 1.3k 5 评论

前端

一个gis瓦片切片转换开源工具

nick_dang 1年前 4.5k 6 评论

Python

前端瓦片地图加载之塞尔达传说旷野之息

dragonir 4年前 5.8k 61 6

前端 JavaScript

WebGIS期末复习

夏辰 4年前 850 2 评论

GIS

1.Electron-封装地图瓦片下载小工具(MapDownload) [配置开发环境]

weshmily前端 4年前 714 点赞 1

前端

矢量切片简介及前端加载

Mapmost 1年前 483 1 评论

前端 GIS

优化实战 第 64 期 - 高德地图千百万数据秒级上图

丢丢哥 2年前 3.2k 11 7

前端 性能优化 GIS

前端地图分类（包括坐标系，GIS基础知识，2D与3D地图结构划分）

陈陈c 3年前 821 3 评论

前端

【WebGIS】OGC API

moonless 1年前 321 1 评论

GIS

地图

dengfenglai 2年前 2.2k 点赞 评论

前端

webGIS开发前端地图

后知后觉就是我 3年前 827 1 评论

GIS

基于Vue+Leaflet实现自己制作一份切片游戏地图

mrbean455 3年前 4.3k 26 12

前端

WebGIS基础理论|地图服务规范



登录掘金领取礼包

更多登录后权益等你解锁



简单地图应用-开源地图OpenStreetMap仿Mapbox样式部署本地矢量瓦片（上）

冰樱湛蓝

1年前

👁 3.6k

👍 17

💬 7

前端

【GIS】GIS全栈工程师计划01——爆肝3w字带你彻底理解GIS

moonless

1年前

👁 512

👍 6

💬 评论

GIS

web3



登录掘金领取礼包

更多登录后权益等你解锁

